

Duració aproximada: 2 hores

IMPORTANT: Tingueu en compte que tots els exercicis s'han de completar utilitzant estrictament el contingut presentat i practicat fins ara. No es permet l'ús d'ordres o instruccions que encara no s'hagin introduït. Creeu una carpeta a GitHub per a cada secció.

CRIDA DE VARIABLES a LINUX BASH

1. Crea un script que mostri les principals variables globals i tots els arguments del Shell, el valor retornat per l'última ordre executada i el PID del Shell actual.



```
alfred@debian: ~  
GNU nano 7.2 Principals_variables.sh  
#!/bin/bash  
  
echo "VARIABLES_PRINCIPALS"  
echo "Usuari: $USER"  
echo "Directorí actual: $PWD"  
echo "Directorí personal: $HOME"  
echo "Shell actual: $SHELL"  
echo "Nom del sistema: $HOSTNAME"  
echo "Data actual: $(date)"  
  
echo "ARGUMENTS_DEL_SHELL"  
echo "Nombre d'arguments: $#"  
echo "Arguments: $@"  
  
echo "VALOR_ULTIMA_ORDRE"  
echo "ULTIMA_ORDRE: $?"  
  
echo "PID ACTUAL"  
echo "PID: $$"
```

2. Crea un script que demani a l'usuari que introdueixi un nom de variable i el seu valor. Desa-la com a variable personalitzada i després mostra-la amb **echo**.

Consell: Pots utilitzar l'ordre **read** en combinació amb **echo**.

```
alfred@debian: ~  
GNU nano 7.2 Demanar_dades.sh  
#!/bin/bash  
  
echo "INTRODUEIX UN NOM DE VARIABLE I EL SEU VALOR: "  
  
read  
  
Variable=$REPLY  
  
echo "Has introduït la variable: $Variable"
```

3. Crea un script en Bash que mostri la informació següent:
 - a. Demana el nom d'una **variable global** (i. e., **HOME**, **USER**, **SHELL**, **PWD**, **PATH**). I execútala.

```
GNU nano 7.2 Variables_especificques.sh  
#!/bin/bash  
  
echo "Introdueix una variable global"  
  
read Nom_variable  
  
echo "Aqui tens la variable $Nom_variable"  
echo "${!Nom_variable}"  
echo "Codi de retorn: $?"
```

- b. Després, mostra els **arguments** passats a l'script en el moment de l'execució.

```
echo "Arguments passats a l'escript"  
echo "Nombre d'arguments: $#"  
echo "Arguments: $@"  
echo "Codi de retorn: $?"
```

- c. En cada cas, mostra el **valor de retorn** de l'última ordre executada(**\$?**).
 - d. Finalment, ensenya el **PID of the current Shell** (**\$\$**).

```
echo "$$"  
echo "Codi de retorn: $?"
```

Pista: Utilitza `echo` per mostrar variables.

4. Crea un script que agafa **5 arguments** (no fa falta validarlos) i els mostra per pantalla. Al final, també ha de mostrar el **nom del script** (`$0`).

```
GNU nano 7.2 5_arguments.sh
#!/bin/bash

echo "Aquí tens 5 arguments"
echo
echo "Numero d'arguments: $#"
```

root@debian:/home/alfred/bash-scripts-workshop/bash-scripts-workshop/Activitat2# ./5_argument:
.sh argumento1 argumento2 argumento3 argumento4 argumento5
Aquí tens 5 arguments

Numero d'arguments: 5

Arguments: argumento1 argumento2 argumento3 argumento4 argumento5

Nom del script: ./5_arguments.sh

5. Crea un script que agafa un nombre (variable) d'arguments i els mostra **un per línia**. Al final, ha d'indicar **quants arguments s'han utilitzat**. No utilitzis `for` o `if`.

Pista: Utilitza `$#` per rebre el nombre d'arguments.

```
GNU nano 7.2 arguments_per_linea.sh
#!/bin/bash

echo "Nombre total d'arguments: $#"
```

echo "Aquests són els arguments:"

echo "\$1"

echo "\$2"

echo "\$3"

echo "\$4"

echo "\$5"

echo "\$6"

echo "\$7"

echo "\$8"

echo "\$9"

```
root@debian:/home/alfred/bash-scripts-workshop/bash-scripts-workshop/Activitat2# ./arguments_per_linea.sh
Esto son argumentos para el script
Nombre total d'arguments: 6
Aquests són els arguments:
Esto
son
argumentos
para
el
script
```

6. Fes un script que agafa múltiples arguments i ensenya la diferencia entre utilitzar `$*` i `$@`. Enseña l'output dins i fora de les cometes dobles (" ").
- a. Comenta com `$*` tracta tots els arguments com una sola string, mentre `$@` ho separa correctament.

```
GNU nano 7.2                                diferencies.sh
#!/bin/bash

echo "Fora de cometes dobles"
echo "$*"
echo "$@"

echo
echo "Dins de cometes dobles"
echo $*
echo $@

root@debian:/home/alfred/bash-scripts-workshop/bash-scripts-workshop/Activitat2# ./diferencies.sh "argumento 1" "argumento 2" "argumento 3"
Fora de cometes dobles
argumento 1 argumento 2 argumento 3
argumento 1 argumento 2 argumento 3

Dins de cometes dobles
argumento 1 argumento 2 argumento 3
argumento 1 argumento 2 argumento 3
```

He fet l'escript pero no aconseguixo que es vegi d'una manera diferent a cada cas.

7. Crea un script que agafa un nombre variable d'arguments i els mostra per pantalla separats un per un amb diferents línies utilitzant **echo**. Al final, mostra el missatge indicant quants arguments s'han utilitzat.

```
GNU nano 7.2                                mostrar_argumets.sh
#!/bin/bash
echo "$1"
echo "$2"
echo "$3"
echo "$4"
echo "$5"
echo "S'han utilitzat $# arguments."
```

```
root@debian:/home/alfred/bash-scripts-workshop/bash-scripts-workshop/Activitat2# ./mostrar_argumets.sh ho
la esto son argumentos
hola
esto
son
argumentos

S'han utilitzat 4 arguments.
```

8. Crea un script que demana un **usuari** i una **contrasenya**. La contrasenya **no ha de ser visible** mentre escriu. **Atenció:** Tant l'usuari com la contrasenya amagada, s'han d'ensenyar en una sola línia..

Pista: Pots utilitzar la comanda **read**.

```
GNU nano 7.2                                usuari_contrasenya.sh
#!/bin/bash

read -p "Introdueix usuari:" usuario

read -s -p "Introdueix contrasenya:" contrasenya
echo
echo "Usuari: $usuario | Contrasenya: $contrasenya"
```

```
root@debian:/home/alfred/bash-scripts-workshop/bash-scripts-workshop/Activitat2# ./usuari_contrasenya.sh
Introdueix usuari:alfred
Introdueix contrasenya:
Usuari: alfred | Contrasenya: super3
root@debian:/home/alfred/bash-scripts-workshop/bash-scripts-workshop/Activitat2# nano usuari_contrasenya.
sh
```

9. Llegeix el següent script, explica l'output de cada línia i contesta les preguntes:

```
#!/bin/bash

CRACK="oriolorioloriol"

echo "The result of ###ori is" ${CRACK###ori}
echo "The result of #*ori is" ${CRACK#*ori}
echo "The result of %%ol* is" ${CRACK%%ol*}
echo "The result of %ori* is" ${CRACK%ori*}
```

echo "The result of ###ori is" \${CRACK###ori}:

`${CRACK###ori}`: Permet eliminar la part més llarga del principi que coincideixi amb `*ori` deixant només un ori.

En `oriolorioloriol` l'última ori és la tercera; si elimines fins allà, queda només el que va després d'aquesta ori: `"ol"`.

Sortida: ol.

echo "The result of #*ori is" \${CRACK#*ori}:

`${CRACK#*ori}`: elimina la part més curta del principi que coincideixi amb `*ori`, deixant la primera ori

En el nostre cas elimina la primera ori, de manera que queda tot el que segueix: `"olorioloriol"`.

Sortida: orioloriol.

echo "The result of %%ol* is" \${CRACK%%ol*}:

`${CRACK%%ol*}`: elimina la part més llarga del final que coincideixi amb `ol*`

En `oriolorioloriol` la primera ol comença després dels tres primers caràcters; si treus des d'aquell ol fins al final, et queda només el prefix abans d'aquell ol: `"ori"`.

Sortida: ori.

echo "The result of %ori* is" \${CRACK%ori*}:

`${CRACK%ori*}`: elimina la part més curta del final que coincideixi amb `ori*`

Això deixa la part abans de l'última ori: `"orioloriol"`.

Sortida: orioloriol.

a. Què fa exactament `##`, `#`, `%%`, i `%`?

`#` : elimina la coincidència més curta des del principi

`##` : elimina la coincidència més llarga des del principi

`%` : elimina la coincidència més curta des del final

`%%` : elimina la cadena més llarga des de el final

b. Com poden ser útils aquestes tècniques a scripts Bash?

Poden ser útils per obtenir nom de fitxers, també per obtenir directoris, eliminar prefixos repetitius.

c. Prova a modificar el valor de `CRACK` i observa com canvia el resultat final.

```
GNU nano 7.2                                CRACK.sh
#!/bin/bash

CRACK="alfredalfredalfred"

echo "The result of ##*ori is" ${CRACK##*ori}
echo "The result of #*ori is" ${CRACK#*ori}
echo "The result of %%ol* is" ${CRACK%%ol*}
echo "The result of %ori* is" ${CRACK%ori*}
```

```
root@debian:/home/alfred/bash-scripts-workshop/bash-scripts-workshop/Activitat2# ./CRACK.sh
The result of ##*ori is alfredalfredalfred
The result of #*ori is alfredalfredalfred
The result of %%ol* is alfredalfredalfred
The result of %ori* is alfredalfredalfred
```

10. Crea un script que mostri un petit “informe del sistema” amb la informació següent:

1. Nom de l'usuari (`$USER`) i el resultat de `whoami`.
2. El directori personal (`$HOME`) i el directori actual (`$PWD`).
3. El contingut del directori actual (`ls`).
4. Quants elements hi ha en aquest directori (`ls | wc -l`).
5. El PID del Shell (`$$`) i el codi de retorn de l'última ordre (`$?`).

```
GNU nano 7.2                               informe.sh
#!/bin/bash

echo "Usuari: $USER"
echo "whoami: $(whoami)"
echo
echo "Directori personal: $HOME"
echo "Directori actual: $PWD"
echo
echo "Contingut del directori actual:"
echo "$(ls)"
echo
echo "Nombre d'elements: $(ls | wc -l)"
echo
echo "PID del shell: $$"
echo
echo "Codi de retorn de l'última ordre: $?"
```

```
root@debian:/home/alfred/bash-scripts-workshop/bash-scripts-workshop/Activitat2# ./informe.sh
Usuari: alfred
whoami: root

Directori personal: /root
Directori actual: /home/alfred/bash-scripts-workshop/bash-scripts-workshop/Activitat2

Contingut del directori actual:
5_arguments.sh
arguments_per_linea.sh
CRACK.sh
Demanar_dades.sh
diferencies.sh
informe.sh
mostrar_argumets.sh
Principals_variables.sh
usuari_contrasenya.sh
Variables_especificques.sh

Nombre d'elements: 10

PID del shell: 14063
Codi de retorn de l'última ordre: 0
```


11. Crea un script anomenat `analitza_fitxer.sh` que faci el següent:

Mostri per pantalla:

```
Analitzant fitxer...
```

1. Demani el nom d'un fitxer amb `read`.
2. Desi en un fitxer `resultats.log`:
3. El nom del fitxer.
4. El nombre de línies útils (no buides).
5. El nombre total de paraules.
6. El nombre de caràcters.
7. La data.
8. Si el fitxer no existeix o hi ha errors, redirigeixi els missatges d'error a `errors.log`.

Si el fitxer no existeix o hi ha errors, redirigeixi els missatges d'error a `errors.log`.

Exemple d'execució:

```
$ ./analitza_fitxer.sh
Analitzant fitxer...
Nom del fitxer a analitzar:
notes.txt
Procés completat! Revisa resultats.log i errors.log
=====
Fitxer analitzat: notes.txt
Línies útils: 23
Paraules totals: 178
Caràcters totals: 1020
Data: Tue Oct  8 12:12:45 CEST 2025
Executat per: alumne1
Directorí: /home/alumne1/scripts
PID Shell: 14192
=====
```

```
GNU nano 7.2                                analitza_fitxer.sh
#!/bin/bash

echo "Analitzant fitxer..."
echo -n "Nom del fitxer a analitzar: "
read fitxer

echo "Procés completat! Revisa resultats.log i errors.log"

{
echo "====="
echo "Fitxer analitzat: $fitxer"
echo "Línies útils: $(grep -cv '^$' "$fitxer" 2>> errors.log)"
echo "Paraules totals: $(wc -w < "$fitxer" 2>> errors.log)"
echo "Caràcters totals: $(wc -m < "$fitxer" 2>> errors.log)"
echo "Data: $(date)"
echo "Executat per: $USER"
echo "Directorí: $PWD"
echo "PID Shell: $$"
echo "====="
} > resultats.log
```

```
|=====
Fitxer analitzat: CRACK.sh
Línies útils: 6
Paraules totals: 30
Caràcters totals: 220
Data: vie 10 oct 2025 20:20:30 CEST
Executat per: alfred
Directorí: /home/alfred/bash-scripts-workshop/bash-scripts-workshop/
Activitat2
PID Shell: 14788
=====
```